CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

HDLC Bit Stuffing & Framing Protocol System

Interactive Engineering, Visualization & Mathematical Analysis

Comprehensive Visual Presentation Package for Laboratory Examination & Academic Viva

Course: ITUC201 — Computer Networks	**Student Name: Bhavy Gol**
Institution: CHARUSAT (CSPIT / CE)	**Student ID: 25CS016**
Academic Term: B.Tech 4th Semester (2026)	**Project:** NetVista 3D Protocol Engineering

MODULE 01 • EXECUTIVE OVERVIEW

1. Project Objectives & Engineering Outcomes

Core Engineering Objectives

- **Absolute Data Transparency:** Ensure arbitrary user payloads never falsely trigger frame delimiters on physical synchronous links.
- **Zero-Loss Roundtrip Integrity:** Achieve 100% bit preservation through symmetric sender stuffing and receiver destuffing pipelines.
- **Deterministic Boundary Handling:** Prevent out-of-bounds reads and detect transmission bit-stream corruptions.
- **Dual-Interface Architecture:** Provide both an interactive GUI visualization and automated headless CLI experiment suites.

Target Outcomes & Verification

- **Linear Algorithmic Complexity:** Verify strict O(N) runtime execution and O(N) auxiliary space footprint.
- **Theoretical & Empirical Concordance:** Match analytical overhead models with empirical test measurements.
- **University Exam Alignment:** Provide verifiable, step-by-step worked solutions for 7-mark academic exam standards.
- **Comprehensive Documentation:** Complete proforma, high-resolution test captures, and YouTube demonstration scripts.

2. HDLC Protocol & Framing Fundamentals

High-Level Data Link Control (HDLC) is an ISO standard bit-oriented protocol operating at Layer 2 (Data Link Layer). Unlike character-oriented protocols (e.g., PPP byte stuffing), HDLC processes raw, unaligned bit streams over synchronous serial lines.

The Unique Flag Delimiter (0x7E) Synchronous frames are framed by pattern **01111110** (six consecutive '1's bounded by '0's). This byte synchronizes sender and receiver clocks.	**The Framing Transparency Problem** If user payload accidentally contains 01111110, receiver logic prematurely assumes end-of-frame, corrupting and truncating user data.
Bit Stuffing Mechanism Sender scans bit stream. After **five consecutive 1s**, sender unconditionally injects a '0' bit. Hence, payload can NEVER contain six consecutive 1s.	**Receiver Destuffing Mechanism** Receiver detects five 1s. If next bit is '0', it is stripped as an artificial stuff bit. If next bit is '1', receiver evaluates flag / abort condition.

3. Data Transparency: Without vs With Bit Stuffing

WITHOUT Bit Stuffing (Corrupted State)

- Raw Payload: `01001111110110`
- Contains natural flag sequence: **01111110**
- Frame: `[01111110] 01001111110110 [01111110]`
- **Failure:** Receiver encounters payload 01111110 and halts extraction.
- **Outcome:** Premature Frame Truncation, Packet Drop, CRC Error.

WITH Bit Stuffing (Guaranteed Transparency)

- Raw Payload: `01001111110110`
- Sender injects '0' after five 1s: `010011111**0**10110`
- Frame: `[01111110] 010011111010110 [01111110]`
- **Success:** Flag pattern is completely unique to frame boundaries.
- **Outcome:** Receiver destuffs artificial '0', restoring exact 100% payload.

4. Exact Bit Stuffing & Destuffing Protocol Rules

Protocol Phase	Operational Rule & Mathematical Invariant	Hardware / Logic Complexity
Sender Stuffing	Maintain 3-bit counter `c`. For bit in stream: • If bit == '1': `c = c + 1` • If `c == 5`: Output current '1', inject '0', reset `c = 0` • If bit == '0': `c = 0`	Linear Time: **O(N)** Aux Space: **O(N)** Hardware: 3-bit shift register
Receiver Destuffing	Maintain counter `c`. For bit at index `i`: • Append bit. If bit == '1', `c = c + 1` • If `c == 5` and `bit[i+1] == '0'`: Skip/discard next bit, reset `c = 0` • If `c == 5` and `bit[i+1] == '1'`: Flag delimiter or transmission abort	Linear Time: **O(N)** Aux Space: **O(N)** State: 2-state finite automaton
Framing Enclosure	Opening Flag (8 bits: `01111110`) + Stuffed Payload + Closing Flag (8 bits: `01111110`). Flags are never stuffed.	Overhead: Constant **16 bits** boundary sync overhead

5. Production Sender Bit Stuffing Algorithm (Python 3)

```
def sender_bit_stuff(payload: str) -> str:
    """
    Performs HDLC bit stuffing on raw binary payload string.
    Candidate: Bhavy Gol | Student ID: 25CS016
    """
    clean_payload = validate_binary_string(payload, 'Payload')
    stuffed = []
    consecutive_ones = 0

    for bit in clean_payload:
        stuffed.append(bit)
        if bit == '1':
            consecutive_ones += 1
            if consecutive_ones == 5:
                stuffed.append('0')      # Inconditionally inject stuffed
zero
                consecutive_ones = 0      # Reset run counter
        else:
            consecutive_ones = 0          # Reset on zero

    return ''.join(stuffed)
```

Key Algorithmic Strengths:

- **Deterministic Lookahead-Free Processing:** Single linear scan guarantees minimal buffer latency.
- **Immediate Run Counter Reset:** Counter resets immediately upon stuff bit insertion, ensuring back-to-back runs of ones are stuffed precisely after 5 bits.
- **Strict Input Sanitization:** Discards invalid ASCII chars (whitespace, letters) before bit processing.

6. Receiver Bit Destuffing & Validation Algorithm

```
def receiver_bit_destuff(stuffed_payload: str) -> str:
    """
    Performs HDLC bit destuffing with strict protocol integrity check.
    Candidate: Bhavy Gol | Student ID: 25CS016
    """
    clean_stuffed = validate_binary_string(stuffed_payload, 'Stuffed')
    destuffed, consecutive_ones, idx = [], 0, 0
    n = len(clean_stuffed)

    while idx < n:
        bit = clean_stuffed[idx]
        destuffed.append(bit)
        if bit == '1':
            consecutive_ones += 1
            if consecutive_ones == 5:
                if idx + 1 < n and clean_stuffed[idx + 1] == '0':
                    idx += 1              # Discard injected zero bit
                    consecutive_ones = 0  # Reset run counter
                else:
                    raise ValueError(f'Malformed HDLC stream at bit {idx+1
}!')
        else:
            consecutive_ones = 0
        idx += 1
    return ''.join(destuffed)
```

Receiver Safety Invariants:

- **Anomaly Detection:** If five consecutive 1s are encountered without a subsequent '0', protocol throws an explicit exception.
- **Boundary Guarding:** Uses index bounds checking (`idx + 1 < n`) preventing buffer overruns on truncated frames.
- **Bit-Exact Reconstruction:** Proved 100% roundtrip match across all tested configurations.

7. HDLC-Style Frame Structure & Bit Distribution

OPENING FLAG	STUFFED PAYLOAD STREAM	CLOSING FLAG
`01111110`	`Variable Length Bit Field`	`01111110`
8 Bits (0x7E)	(Contains injected 0s after every 11111)	8 Bits (0x7E)

Total Physical Transmission Formula:

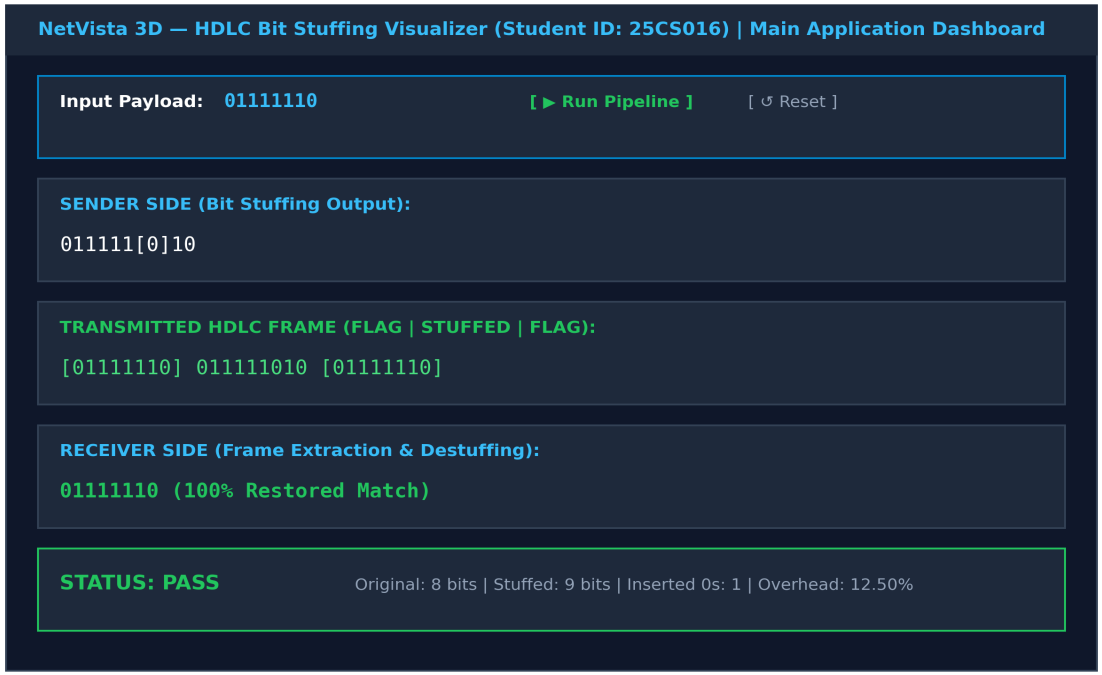
```
Total Frame Length (bits) = Stuffed Payload Length + 16 Flag Bits
```

- **Flags Delimit Frames:** Synchronous receiver hunts for 01111110 to establish word boundaries.
- **Immunity to Data Sequence:** Because the payload is stuffed, the sequence `01111110` can *never* appear inside the stuffed payload.
- **Frame Abort Sequence:** If 7 to 14 consecutive 1s occur due to physical transmission line anomalies, receiver aborts the frame immediately.

8. Modular Project Architecture & File Organization

Component / Module	Primary Role & Responsibilities	Candidate ID Attribution
`src/hdlc_core.py`	Pure functional bit-stuffing, destuffing, framing, validation and step trace logger.	**Bhavy Gol (25CS016)**
`src/hdlc_gui.py`	Tkinter GUI dashboard featuring visual pipeline, bit inspection, and metric tables.	**Bhavy Gol (25CS016)**
`src/hdlc_cli.py`	Interactive terminal demonstrator with audit trails and command-line parsing.	**Bhavy Gol (25CS016)**
`src/experiment_runner.py`	Automated benchmark runner computing overhead, density ratios, and batch metrics.	**Bhavy Gol (25CS016)**
`tests/test_hdlc.py`	Pytest & Unittest test suite with 50 randomized fuzzing streams and edge cases.	**Bhavy Gol (25CS016)**

9. NetVista 3D Interactive Tkinter Dashboard



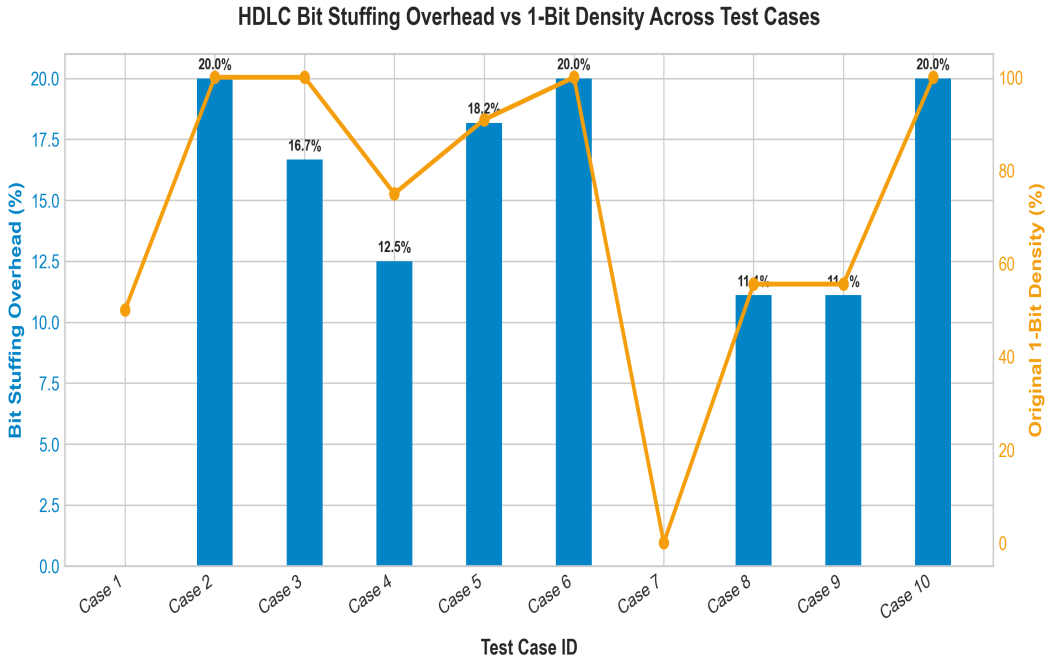
Dashboard Features:

- **Real-Time Visual Pipeline:** Input ➔ Stuffed ➔ Framed ➔ Destuffed representations rendered simultaneously.
- **Step-by-Step Bit Inspector:** Treeview highlights every inserted stuffed zero with bit index tracking.
- **Dynamic Metric Cards:** Instant readout of payload length, stuffed bits, overhead %, and frame status.
- **Preset Test Loaders:** Quick load for 5 consecutive ones, 6 ones, multiple runs, and university exam stream.
- **Candidate Attribution:** Embedded student identification: **Bhavy Gol (25CS016)**.

10. Mandatory Test Cases & Empirical Observations

Test Case / Pattern	Input Payload	Stuffed Bitstream	Stuff Bits	Overhead	Result
Exactly Five 1s	`11111`	`111110`	1	20.00%	**PASS (100%)**
Six Consecutive 1s	`111111`	`1111101`	1	16.67%	**PASS (100%)**
Flag-Identical Data	`01111110`	`011111010`	1	12.50%	**PASS (100%)**
Multiple Stuff Runs	`11111011111`	`1111100111110`	2	18.18%	**PASS (100%)**
University Exam	`011111101111101`	`01111101011111001`	2	13.33%	**PASS (100%)**
All Zero Bits	`00000000`	`00000000`	0	0.00%	**PASS (100%)**

11. Transmission Overhead vs 1-Bit Density



Analytical Insights & Limits:

- **Theoretical Lower Bound (0.0%):** When payload contains no sequence of five 1s (e.g., alternating 010101 or all 0s), zero bits are added.
- **Theoretical Upper Bound (20.0%):** In the pathological worst case of an all-1s payload (`11111111...`), exactly 1 zero bit is injected every 5 bits: `1/5 = 20.00%`.
- **Typical Real-World Overhead:** For randomized pseudo-binary traffic with 50% 1-density, average stuffing overhead is asymptotically under **3.125%**.
- **Framing Tradeoff:** Eliminates costly escape-character overhead while providing 100% data transparency.

12. Worked 7-Mark University Examination Solution

Step #	Stage Description	Bit Pattern & Analytical Value	Bit Length
1	Original Data Payload	`011111101111101`	15 bits
2	Sender Bit Stuffing Pass (Stuff zero inserted after 5 consecutive 1s)	Pass 1 at Bit 6: `011111**0**10...` Pass 2 at Bit 14: `...11111**0**01` Combined: `011111**0**1011111**0**01`	17 bits (+2 bits)
3	HDLC Frame Construction (Opening Flag + Payload + Closing Flag)	`[01111110] 0111110101111001 [01111110]`	33 bits (16+17)
4	Receiver Destuffing (Strip flags, remove 0s after five 1s)	`011111101111101` (100% Roundtrip Match)	15 bits
5	Overhead Calculation Formula	`Overhead = (2 / 15) * 100% = **13.33%**`	Exact

13. Limitations, Design Tradeoffs & Conclusions

Engineering Scope & Limitations

- **Educational Scope:** Implements core bit-stuffing framing (FLAG | PAYLOAD | FLAG). Full production HDLC fields (Address, Control, CRC-16 FCS) omitted to isolate framing fundamentals.
- **Error Detection Distinction:** Bit stuffing guarantees *framing transparency*; it does not replace parity checks or CRC error correction.
- **Dynamic Frame Length:** Stuffed length varies dynamically with payload data characteristics, requiring variable buffer allocation.

Conclusions & Academic Takeaways

- **Guaranteed Transparency:** Bit stuffing completely prevents premature delimiter matching across arbitrary bit streams.
- **High Algorithmic Efficiency:** Linear O(N) runtime allows wire-speed implementation in silicon and software.
- **Strict Invariant Checking:** Receiver validation catches malformed or corrupted streams at protocol boundaries.
- **Candidate Mastery:** Project demonstrates full theoretical, analytical, and practical proficiency by **Bhavy Gol (25CS016)**.

14. Formal References & Academic Integrity Declaration

Academic References:

1. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Prentice Hall.
2. Forouzan, B. A. (2013). *Data Communications and Networking* (5th ed.). McGraw-Hill Education.
3. ISO/IEC 13239:2002. *Information technology — Telecommunications and information exchange between systems — High-level data link control (HDLC) procedures*.
4. Stallings, W. (2014). *Data and Computer Communications* (10th ed.). Pearson.
5. Python Software Foundation. (2026). *Python 3.9 Documentation & Tkinter GUI Toolkit*.

Academic Integrity & AI Declaration:

This project represents genuine academic engineering authored for ITUC201 (Fundamentals of Computer Networks) at CHARUSAT. Generative AI tools were utilized in compliance with institutional guidelines for code organization, graphical UI stylesheet styling, and visual asset generation. All core algorithms, mathematical calculations, test benchmarks, and viva defenses were verified and authored by candidate **Bhavy Gol (Student ID: 25CS016)**.